

Layer Distinction 2 — CKS Is Not an Agent Framework: Standalone Treatment of the Architectural Boundary Between CKS and Agent Frameworks (Multi-Agent Libraries Built Around LLM Execution) in CKS

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 5 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the second of three orchestration-layer distinctions — the boundary between CKS and **agent frameworks** (multi-agent libraries built around LLM execution) — as a standalone architectural commitment with independent operational content, separable from the CKS-vs-workflow-engine and CKS-vs-control-plane boundaries and from the broader cross-layer composition pattern with which it composes. The treatment **extends** the source paper's layer-distinction framing — the substrate-vs-cell split at §2.1, the AI-as-substrate-mediator commitment at §4.2, and the MACI treatment at §4.4 — to the agent-framework design-object family the source paper does not directly name. The extension is explicit; the source paper's framing covers the underlying layer model and the AI-mediator commitment, and this note applies that framing to the agent-framework class specifically.

Abstract

The Coordination Knowledge Substrate (CKS) pattern is at architectural risk of being conflated with agent frameworks — the multi-agent libraries built around LLM execution that source paper §4.4 treats through MACI as canonical 2024–2026 example, including LangChain, LlamaIndex, AutoGen, CrewAI, and the broader category. The two surfaces look superficially similar in a way that distinguishes this conflation from the CKS-vs-workflow-engine boundary: both store content related to LLM operations, both can be read across sessions, both can grow over time. The architectural commitments differ on every load-bearing axis, but the surface similarity makes the conflation easy and commercially consequential. This note formalizes the boundary as standalone architectural commitment with four operational components — different state ownership, different lifecycle, different read/write access, different architectural role — plus two sharpening properties (human-direct-modification, lifecycle-independence) that operationalize the test. It states what the boundary does and does not claim, distinguishes it from four adjacent agent-framework variations, enumerates ten failure modes that violate it specifically, and provides an operational test for whether a system distinguishes its coordination state architecturally from agent-framework execution state.

1. Why the CKS-vs-agent-framework boundary needs to be formalized as standalone

A prior foundational note formalizes the three orchestration-layer distinctions jointly, and an integrating-frame operational note recapitulates them at full operational scope. A separate specialization formalizes the CKS-vs-workflow-engine boundary as having independent architectural content. This note formalizes the CKS-vs-agent-framework boundary in the same way, with particular weight, because the conflation it addresses is the most commercially consequential of the three.

The motivating cases are concrete: CKS deployments coordinate with agent frameworks in hybrid systems — LangChain or LlamaIndex agents executing cells, AutoGen or CrewAI managing multi-agent cell execution, enterprise agent platforms with persistent memory backing cell runs, vendor-provided agent runtimes invoked by cells. Each scenario requires the layer distinction to be operationally specified, particularly because the framework's memory may operationally appear substrate-like even when it carries none of the substrate's architectural commitments.

The strategic prior-art posture is the second motivation. Agent frameworks are the dominant 2024–2026 LLM-coordination pattern, with extensive prior work in multi-agent libraries, persistent agent-memory architectures, and multi-agent reasoning protocols. Patentable derivations focused on AI-coordination architectures with agent integration, multi-agent reasoning systems with persistent state, or LLM-driven coordination architectures are substantially more defensibly contested when the boundary is publicly formalized as standalone.

The third motivation is the load-bearing connection to the AI-as-substrate-mediator commitment at source paper §4.2. The mediator decomposition's Property C — the LLM does not hold substrate-relevant state outside the substrate — is directly load-bearing for this boundary. Agent-framework memory is exactly the kind of LLM-held state outside substrate that Property C addresses when the framework's memory is treated as substrate. Without the boundary specified standalone, the mediator commitment fails specifically at the agent-framework integration point. The fourth motivation, related, is the explicit-extension framing: the source paper's framing at §2.1, §4.2, and §4.4 covers the underlying layer model and mediator commitment; this note applies that framing to the agent-framework design-object family the source paper does not directly name, with the extension named explicitly so downstream readers understand the citation relationship.

2. Agent frameworks, defined precisely

In the architectural literature of 2024–2026, **agent frameworks** are coordination-mechanism-layer objects specialized for LLM-driven execution. The category has four defining properties.

The design object is an LLM-execution coordinator. The framework defines how LLM agents communicate, how planning and reflection are handled, how tool calls are routed, and how multi-step reasoning is structured into a system-level result. The architectural concern is *how the LLMs run*.

The design commitment includes memory as part of the execution machinery. Memory typically takes one or more of several forms: per-session conversation state, per-agent scratchpads, vector stores over past interactions, summary stores for long-running agents, and shared memories accessible across agents inside a single framework instance. Memory is architecturally part of what the framework provides; it is not an adjacent component the framework consults.

The category includes the dominant multi-agent libraries. LangChain, LlamaIndex, AutoGen, and CrewAI are coordination-mechanism-layer objects despite differing memory and multi-agent semantics. MACI, which §4.4 treats as canonical, is in the same category, as are enterprise agent platforms with persistent state. The category is defined by the commitment to LLM-execution semantics, not by the specific memory or multi-agent mechanism.

The architectural commitment is to LLM-execution semantics and to execution state. Memory is part of execution state; it persists for the duration of agent runs and may be archived or summarized at framework shutdown, but its persistence is conditional on framework configuration rather than architectural commitment. Agent frameworks address a real and important architectural concern — running multi-agent LLM systems reliably and observably — at a different layer from the one CKS commits at.

3. The CKS-distinguishing commitment, defined precisely

The CKS-vs-agent-framework boundary has four operational components, each corresponding to a load-bearing architectural axis on which CKS substrates and agent-framework memory differ. A system distinguishes CKS from an agent framework architecturally only when all four components hold.

(a) Different state ownership. CKS substrate is governed by humans per A1.01 and is read by humans exercising governance per the three rights at A2.01–A2.03 and by cells executing under human-authored orchestration rules per A2.04. The substrate's content is humans' coordination artifact: decisions humans have made or have authorized cells to make under their rules, rationale humans understand, conflicts humans have chosen to preserve. Agent-framework memory, by contrast, is *the agent's* — organized for the agent's consumption, written and read primarily by the agent or by other agents inside the same framework, and shaped by the framework's data model rather than by human governance authority. A substrate readable by humans but architecturally owned by agents is not CKS substrate.

(b) Different lifecycle. CKS substrate persists coordination state independent of any agent's lifecycle. By architectural commitment, substrate content survives every agent's termination and every framework instance's lifecycle. Agent-framework memory is bound to the agent's lifecycle: when the agent terminates, the memory is discarded, archived in a framework-level store, or summarized into a memory representation that lives at the framework's layer. The lifecycle commitment differs in kind, not in degree: the substrate's persistence is architectural — the substrate is the persistent coordination artifact, definitionally; framework memory's persistence is configuration- or convention-dependent. A framework configured to retain memory indefinitely is still framework memory.

(c) Different read/write access. CKS substrate is human-inspectable and human-modifiable in the tools it is hosted in, with the three rights per A2.01–A2.03 exercisable directly. The inspect right does not require LLM intermediation as precondition; the modify right operates on substrate content directly; the override right is exercisable at the human's chosen time per the temporal property at A2.07. Agent-framework memory sits behind the framework's interface and is shaped by the framework's data model; humans accessing it typically do so through framework APIs, framework tools, or framework-mediated interfaces. The two access models differ architecturally even when the framework happens to expose convenient inspection tools, because the architectural commitment is to whether the rights are exercisable as a property of the system's design, not as a feature of the framework's current interface.

(d) Different architectural role. CKS substrate is *coordination state that humans govern* — its persistence and addressability are architectural commitments. Substrate changes are addressable per A2.52 (Guarantee C of the determinism contract); the substrate is the source of truth for coordination questions per A2.41 (the five categories of authoritative state: decisions, by-whom, authority, rationale, contradictions). The role is governance-bearing. Agent-framework memory is *execution state that happens to persist* — persistence is a feature of memory, not a commitment to coordination governance; the role is execution support. The AI-as-substrate-mediator commitment per A1.04 specifies that LLMs do not hold substrate-relevant state outside the substrate (Property C per A2.21); agent-framework memory operating as substrate substitute violates this directly.

The four components together specify the architectural distinction. A system that satisfies all four has the boundary in the architectural sense; a system that satisfies fewer is not CKS-coherent on the agent-framework axis.

4. What the boundary does NOT claim

The standalone treatment is bounded. Stating precisely what the boundary does not claim is what keeps it from drifting into something stronger than the source paper supports.

It does not claim that agent frameworks are inferior to CKS. Agent frameworks address LLM-execution coordination; CKS addresses coordination-knowledge state. The two design objects serve different design goals at different architectural layers. The boundary specifies architectural distinction at the layer-commitment level, not architectural superiority.

It does not foreclose agent-framework-executes-CKS-cell hybrid compositions. Per A1.16 and a separate composition-pattern treatment, an agent framework may execute a CKS cell: the framework manages the LLM execution machinery, and the cell reads from and writes to the substrate under human-authored orchestration rules per A2.04.

It does not require CKS to operate without LLM-execution machinery. CKS cells use LLMs per A1.04 and the mediator decomposition; the architectural commitment is that LLMs operate as substrate mediators, not that LLM-execution machinery is absent.

It does not foreclose framework memory features or CKS-style features in agent frameworks. Specific frameworks may include persistent memory, vector-store memory backends, summarization-based memory, or cross-session memory APIs; they may also include human-readable memory inspection surfaces, governance UIs, or audit logs over memory access. The architectural commitment is to whether all four components per §3 hold at the architectural level — not to which operational features happen to be present. Operational features that approach CKS-style content do not transform an agent framework into CKS; treating framework memory as substrate substitute violates the boundary regardless of how convenient the inspection tooling is.

It does not specify implementation patterns for cell-execution. Specific composition primitives — substrate-aware agent memory interfaces, framework-substrate read/write contracts, cell-execution APIs — are explicitly future work per source paper §13.3.

5. What the boundary is NOT

Four agent-framework variations are commonly conflated with the boundary. Each is a real and reasonable variation in some agent-framework architecture; naming what the boundary is not prevents the misreading.

Not frameworks with persistent memory. Frameworks that persist memory across agent runs — long-term memory stores, persistent agent state — extend memory's lifecycle but remain agent frameworks architecturally. Persistence is a memory feature in execution architecture; the substrate's persistence is architectural with humans-as-governors. Adding persistence does not transform framework memory into CKS, because the four components per §3 remain unchanged.

Not frameworks with vector-store memory backends. Frameworks backing memory with vector stores — similarity-based retrieval, embedding-based agent memory — operate on memory as similarity-indexed agent state. Vector backing is an implementation choice; CKS substrate is coordination content with substrate-resident provenance and human-governed authority. Storage technology does not change architectural commitment.

Not frameworks with summarization-based memory. Frameworks summarizing long-running agent state — running-summary memory, hierarchical memory — extend memory's scalability but remain agent frameworks architecturally. Summarization compresses agent state; substrate content is decisions, rationale, and authority, not compressed agent memory.

Not frameworks with cross-session memory APIs. Frameworks exposing memory across sessions through APIs — cross-session conversation continuity, multi-session agent memory — extend memory's accessibility but remain agent frameworks architecturally. Cross-session APIs operate on agent memory through framework mediation; CKS substrate is human-inspectable and human-modifiable directly.

6. Why the boundary is load-bearing for downstream commitments

The boundary is load-bearing for several CKS commitments. For the AI-as-substrate-mediator commitment per A1.04 and the mediator decomposition A2.18–A2.23: Property C per A2.21 is directly violated when agent-framework memory operates as substrate substitute. For the human-governed commitment per A1.01 and the three rights per A2.01–A2.03: substrate is exercisable per the three rights directly; framework memory is exercisable only through framework mediation. For the temporal property per A2.07: substrate is governable at any moment; framework memory is governable only at framework-permitted moments. For the substrate-as-source-of-truth commitments per A2.41: substrate is authoritative for the five categories of authoritative state; framework memory is not. For the hybrid systems composition framework per A1.16: the boundary is what makes agent-framework-executes-CKS-cell composition coherent rather than a fused architecture in which the layers cannot be distinguished.

7. Failure modes that violate the boundary

A system can fail the boundary specifically, even when it satisfies the workflow-engine and control-plane boundaries and broader CKS commitments. Ten failure modes name the most common ways. The first three are weighted: in commercial 2024–2026 deployments labelled “agentic AI,” they are the patterns most often observed.

(a) Agent-framework-memory-as-substrate. The implementation treats agent-framework memory as the system's coordination substrate. Component (a) — state ownership — fails: the memory is the agent's, not humans' coordination artifact. This is the canonical failure mode and the one most commonly produced by commercial pressure to deliver “agent memory” capabilities while claiming substrate-equivalent governance properties. The system carries rich state during agent runs and loses coordination state when agents terminate.

(b) Persistent-agent-memory-as-coordination-knowledge. The implementation treats persistent agent memory — long-term memory stores, durable agent state — as coordination-knowledge state. Component (d) — architectural role — fails: persistence is a memory feature in execution architecture, not a commitment to coordination governance. The failure is harder to detect than (a) because the persistence is real; what is missing is the governance commitment.

(c) Framework-mediated-governance. The implementation requires humans to access substrate-equivalent content through framework APIs, framework tools, or framework-mediated interfaces exclusively. Component (c) — read/write access — fails: substrate is human-inspectable and human-modifiable directly. Convenient inspection tools do not satisfy the right architecturally if they are the only path.

(d) Lifecycle-bound-coordination-state. Coordination-knowledge state is bound to agent or framework lifecycle. Component (b) fails.

(e) LLM-held-substrate-state. The LLM in agents holds substrate-relevant state outside the substrate (working context contains substrate-equivalent decisions; agent memory carries coordination content not present in substrate). Property C per A2.21 fails directly.

(f) Vector-memory-as-substrate. Vector-store agent memory is treated as substrate. The vector store operates on similarity-indexed agent state; substrate operates on coordination content with provenance and human authority. Components (a) and (d) fail.

(g) Summarized-memory-as-coordination-state. Summarized agent memory is treated as coordination state. Summarization compresses agent state; coordination state is human-governed content with substrate-resident provenance. Component (d) fails.

(h) Cross-session-memory-as-substrate-equivalent. Cross-session agent memory is treated as substrate-equivalent. Cross-session APIs operate on agent memory through framework mediation; substrate is architecturally addressable across sessions per A2.52 with direct human access. Component (c) fails.

(i) Framework-archive-as-substrate-archive. Archived agent memory is treated as the architectural coordination archive. Framework archives operate at the framework's layer; substrate operates at the coordination-knowledge layer. The layer distinction collapses.

(j) Agent-only deployment claiming CKS commitments. The implementation operates only at the agent-framework layer but claims to satisfy CKS commitments through framework features (e.g., “our agents have memory” presented as “we have CKS substrate”). The four components per §3 are not architecturally satisfied; CKS commitments are claimed performatively.

A system that exhibits any of (a)–(j) does not implement the boundary specifically, regardless of how operationally substrate-like the framework's memory may appear.

8. Operational test

A system instantiates the CKS-vs-agent-framework boundary if and only if all of the following are true at all times during the substrate's existence:

1. The system's coordination-knowledge content is held in a substrate humans can directly read and write under governance authority per A1.01 and the three rights per A2.01–A2.03 — not in agent-framework memory accessed through framework mediation.
2. The substrate is addressable across sessions independent of any agent's lifecycle per A2.52 — not in agent-framework memory bound to agent or framework lifecycle.
3. The substrate's content is decisions, rationale, authority, and preserved conflict per the source-of-truth categories at A2.41 — not agent execution state organized for agent consumption.
4. LLMs in cells operate as substrate mediators per A1.04; LLMs do not hold substrate-relevant state outside substrate per Property C per A2.21 — not as autonomous agents holding substrate-equivalent memory.
5. **Human-direct-modification.** A human can modify substrate content directly without framework mediation. If modification requires framework APIs, framework tools, or framework-mediated interfaces exclusively, the content is framework memory, not substrate.
6. **Lifecycle-independence.** Substrate content persists after every agent terminates by architectural commitment, not by archival convention. If persistence is conditional on framework configuration or archival policy, the content is framework memory, not substrate.
7. Hybrid compositions with agent frameworks (per A1.16) explicitly name the layers — the agent framework executes CKS cells; framework memory holds execution state for the cell's run; the CKS substrate persists coordination state across cell runs.

A system that fails any of (1)–(7) does not instantiate the boundary in the architectural sense, even if agent-framework memory operationally appears substrate-like.

The one-sentence test. If a system's “memory” is held inside an agent framework's execution context, organized for agent consumption, and read primarily by agents within that framework, it is agent-framework memory; if the same content is held in a substrate humans can directly read and write under governance authority, addressable across sessions independent of any agent's lifecycle, it is CKS substrate. The one-sentence test names the most operationally distinctive axes — state ownership, lifecycle independence, direct human access — for any specific instance; the four-component specification per §3 plus the sharpening properties at (5) and (6) provide the full definition for cases requiring detailed analysis.

9. Why naming the boundary as standalone matters

Implementations under pressure to deliver “agentic AI” or “AI memory” capabilities consistently drift toward agent-framework-memory framings. The drift is steady because agent frameworks are the dominant 2024–2026 LLM-coordination pattern, agent-framework memory and CKS substrate look superficially similar, audiences understand “the agents have memory” more readily than “the substrate carries coordination state with human-governed authority,” and adding persistent memory to agent frameworks appears as natural evolution rather than as architectural reassignment of state.

Implementations that drift produce systems that hold rich state during agent runs and lose coordination state when agents terminate. The downstream consequences manifest as Property C violations per A2.21, lifecycle-binding failures, human-direct-access failures against the three rights at A2.01–A2.03, source-of-truth failures against the categories at A2.41, and architectural-commitment failures that obscure A1.01, A1.04, and A2.07 simultaneously. Naming the boundary as standalone — with the four operational components, the limitations, the four adjacent-variation distinctions, the load-bearing connections, the ten failure modes, and the operational test with two sharpening properties — gives downstream readers a precise specification of what distinguishes CKS from agent frameworks. The subsequent specialization formalizes Layer Distinction 3 (control planes); a further note formalizes the cross-layer composition pattern.

Subsequent work that adopts the CKS pattern, composes it with agent frameworks, or argues against the boundary should use the CKS-vs-agent-framework distinction in the sense formalized here. Subsequent work that uses the distinction differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Layer Distinction 2 — CKS Is Not an Agent Framework: Standalone Treatment of the Architectural Boundary Between CKS and Agent Frameworks (Multi-Agent Libraries Built Around LLM Execution) in CKS*. 5 May 2026. ORCID: 0009-0004-8065-3235.